

Group names:

- Chuma Nxazonke (Team leader): 219181187
- Ishmail Mgwena: 215088417
- Thabiso Matsaba: 220296006
- Athini Mbiko: 213196379

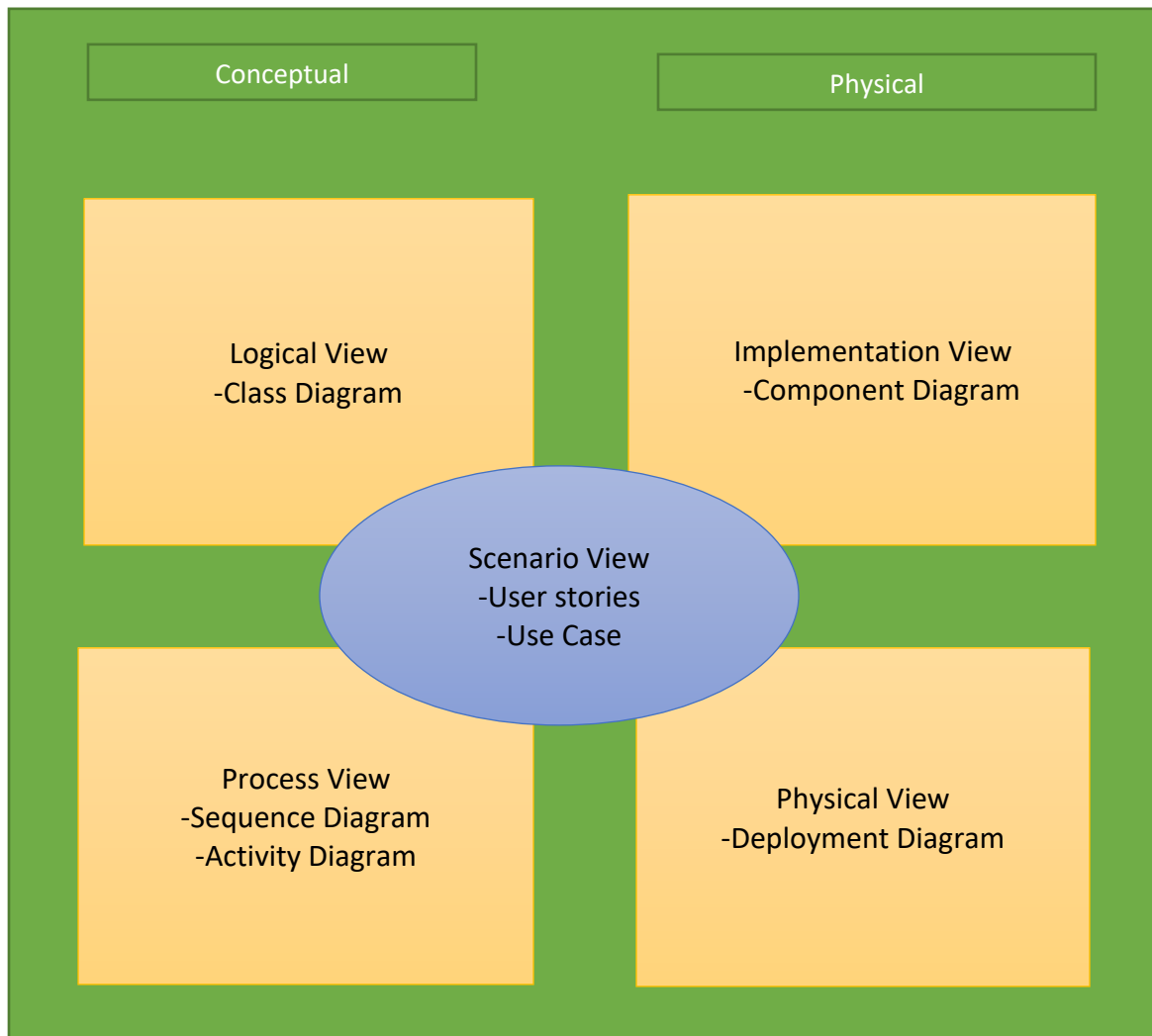
Application Development Theory 3

(ADT372S)

Assignment 3 task 3 no. 5

Due date: 30 April 2022

Architectural Design 4+1 Method



Architectural Design

Client-Server Architecture (3-tier)

The automation of human-performed tasks has taken the business world by storm. As seen by the fast-food giant, McDonald's, which has implemented a system for customers to place orders themselves via integrated multimedia devices, more and more businesses are attempting to integrate information technology into their daily business operations. In order to design a successful restaurant management system, we decided to reuse system architectures used for e-commerce as well as transaction processing systems.

Non-Functional requirements, identified by implementing activity diagrams and found in the Process View, have the biggest impact on the system architecture chosen. Non-Functional requirements considered includes:

- Performance
- Security
- Safety
- Availability
- Maintainability

Because these five non-functional components are critical to the system's success, the architectural design decision was made to implement a Layered Architecture. Multiple applications can reuse the application components that have already been created by using a layered architecture. It also improves the application's safety and security because different parts of the application can reside in different levels of security. Because different teams can work on different layers of the system, it is easier to maintain the system because more people can work to improve the application at the same time.

Choosing the layered architecture will lead to a decrease in performance of the system, yet this is a fair trade for the increase in security, safety, and maintainability. This architecture enables the development of loosely coupled systems, which will increase availability as if one component breaks the rest will still function properly.

The system will be divided into three layers: presentation, application, and database. The Presentation layer will include a User Interface that the customer will use to send requests to the server. This will be accomplished through the use of a browser window. The Application layer will contain all business logic used to process and protect captured data. The Database layer is used to store and archive business data for later analysis.

The system will be comprised of three servers, each of which will provide a different service, namely Business-As-Usual, Order, and Transaction. If a customer requires assistance or wants the bill, the BAU server will ensure that all employees can see their open tables. The Order server will coordinate

customer orders to the kitchen and notify waiters when the food is ready to serve. Customers will use the Transaction Server to ensure that payments are made fairly and honestly.

According to the Logical View, the only entities identified as relevant to the system are the Host/Waiters, Bus Boys, Kitchen Staff, and Management. These entities comprise the class diagram of our restaurant system. The Class Diagram depicts the system as a static structure diagram comprised of object classes with attributes and operations. It also demonstrates the relationship between these object classes and aids in determining the system's functional requirements. Customers and end-users will benefit from this diagram.

The Development view depicts the software as components for development purposes. A single developer or development team is responsible for implementing these components. The Component Diagram is the UML Diagram used to visualize these components.

The Component Diagram depicts the system structure as components and how they are linked to form a larger system. This helps to ensure that planned software development meets the system requirements. This diagram is beneficial to software development teams as well as managers. We also decided to update our use case from the previous use case diagram we created.

Use Cases

